digiSpace

Research Division

Advanced Mathematical Modeling and Algorithmic Enhancements for FL Studio Piano Roll Scripting

A Comprehensive Review of Velocity Curves

Author: Dennis 'dendogg' Smaltz

Artifact Type: Research Report (docs) | Version: 1.0.0 | Date: 2026-03-10
A.I. Acknowledgement: Anthropic - Claude Opus 4.6

Aim Twice, Shoot Once!

Table of Contents

Table of Contents 2
1. Introduction to Algorithmic MIDI Manipulation and the Scripting Ecosystem 3
2. Architectural Critique of the Baseline Implementation 5
2.1 The Normalization of the Time Domain 5
2.2 Arbitrary Curve Functions and Lack of Parameterization 5
2.3 The Naive Implementation of Stochastic Humanization 6
2.4 Rigid Rhythmic Constraints and Metric Ignorance 7
3. Advanced Mathematical Paradigms for Continuous MIDI Automation 8
3.1 Transcending Static Exponentials: The FL Studio Tension Algorithm 8
4. Formulating the S-Curve: The "Double Curve" Architecture 10
4.1 Evaluation of the Lower Hemisphere 10
4.2 Evaluation of the Upper Hemisphere 10
S-Curve Hemisphere Evaluation Summary 11
5. Precision Interpolation via Cubic Bézier Curves 12
5.1 Bernstein Polynomials and Parametric Equations 12
5.2 Numerical Approximation via the Newton-Raphson Method 12
6. Stochastic Biometric Humanization via Gaussian Distributions 14
6.1 The Psychoacoustic Flaw of Uniform Randomness 14
6.2 Implementing the Box-Muller Transform in Native Python 14
6.3 Algorithmic Application to MIDI Data 14
7. Dynamic Polyrhythmic Scaling via Euclidean Algorithms 15
7.1 The Mathematical Limitations of Hardcoded Meters 15
7.2 Integrating Bjorklund's Algorithm for Euclidean Rhythms 15
Bjorklund Algorithm Trace: E(3, 7) 15
7.3 Applying Polyrhythmic Scaling to Velocity Geometries 15
8. API Utilization and UI/UX Refinements 17
8.1 Expanding the ScriptDialog Framework 17
8.2 Structural Exception Mitigation 17
9. Synthesis of the Refactored Algorithmic Pipeline 18
10. Conclusion 19

1. Introduction to Algorithmic MIDI Manipulation and the Scripting Ecosystem

The introduction of native Python scripting capabilities for the Piano Roll in FL Studio 21.1 represented a fundamental paradigm shift in how digital composers, electronic musicians, and audio engineers manipulate Musical Instrument Digital Interface (MIDI) data.[1] Prior to this architectural update, users were largely confined to manual graphical event drawing, rigid quantization grids, or basic macro tools such as the built-in randomizer and riff machine. By exposing the internal piano roll data structures through the flpianoroll Python API, the software environment now permits the programmatic generation, dynamic transformation, and contextual analysis of note properties with virtually limitless complexity.

Among the multitude of note properties exposed by the API—including pan, fine pitch, filter cutoff, and resonance—velocity remains arguably the most critical parameter in the domain of digital orchestration.[2] Velocity determines not only the dynamic impact and amplitude of a synthesized voice but also governs the timbral quality of modern multi-sampled instruments. In high-fidelity sample libraries, such as orchestral strings or acoustic drum kits, varying the MIDI velocity triggers entirely different sample layers, capturing the physical resonance, harmonic excitation, and mechanical noise associated with varying degrees of instrumental force. Consequently, applying static, linear, or uniformly random velocities results in an artificial, robotic auditory experience that immediately breaks the illusion of acoustic realism.

The baseline Python script under review, titled "Velocity Curves," provides a rudimentary computational framework for applying mathematical functions across the time span of selected MIDI notes to govern their velocity.[3] The script successfully leverages the flpianoroll API to instantiate a user interface via the ScriptDialog module, iterates over the selected note objects using the score.noteCount property, and normalizes the temporal positioning of these notes into a parametric variable. However, while the structural boilerplate of the script is functionally sound, the mathematical models, curve topologies, and rhythmic algorithms employed within its processing loop are highly simplistic and mathematically limited.

This comprehensive research report provides an exhaustive architectural, mathematical, and algorithmic review of the baseline velocity script. The objective is to systematically deconstruct its current methodologies and propose rigorous algorithmic enhancements. By integrating advanced mathematical curve formulations—specifically the exact non-linear Tension algorithms utilized natively within the digital audio workstation's automation engine[4] and the implementation of

[1] FL Studio 21.1 Python Piano Roll scripting API. See Image-Line documentation for flpianoroll module reference.
[2] MIDI velocity governs both amplitude and timbral quality in multi-sampled instruments, triggering distinct sample layers.
[3] Baseline script: Velocity Curves for FL Studio Piano Roll, utilizing flpianoroll Python API.
[4] FL Studio internal tension algorithm, attributed to 'gol' (Didier Dambrin), the primary architect of FL Studio.

precise Cubic Bézier interpolation[5]—the script can achieve true professional parity. Furthermore, by replacing uniform stochastic randomness with Box-Muller Gaussian distributions to simulate genuine biometric jitter[6], and by discarding static meters in favor of dynamically generated Euclidean rhythms utilizing Bjorklund's algorithm[7], the script's usefulness, musicality, and mathematical accuracy can be exponentially amplified.

[5] Cubic Bézier curves are the industry standard for motion graphics, CAD, and dynamic physics interpolation.
[6] Box, G.E.P. and Muller, M.E. (1958). 'A Note on the Generation of Random Normal Deviates.' Annals of Mathematical Statistics.
[7] Bjorklund, E. 'The Theory of Rep-Rate Pattern Generation in the SNS Timing System.' (2003). Adapted by Toussaint for music.

2. Architectural Critique of the Baseline Implementation

Before implementing advanced mathematical algorithms, it is logically necessary to deconstruct the current script's methodology to identify structural bottlenecks, computational inaccuracies, and musical limitations. The baseline script is divided into three primary functional blocks: a custom random number generator, a user interface definition, and the main application loop.

2.1 The Normalization of the Time Domain

The fundamental operation required to apply any mathematical curve across a sequence of discrete events is the normalization of the evaluation domain. In the context of the piano roll, this domain is time. The script correctly isolates the temporal boundaries by calculating the minimum and maximum start times of the selected notes, establishing a `time_span` variable. For each note in the selection array, its relative position is normalized into a floating-point parameter *t*:

$$t = \frac{(note.time - min_time)}{time_span}$$

This operation effectively maps the physical timeline of the selected notes to a dimensionless parameter *t* strictly bounded within the interval (0.0, 1.0). This normalization is the foundational prerequisite for evaluating standard parametric curves. Furthermore, the script exhibits defensive programming by verifying that `time_span > 0` before executing the division, mitigating fatal `ZeroDivisionError` exceptions that would crash the Python interpreter if a user applied the script to a single note or a vertical chord cluster where all notes share the exact same temporal onset.

However, a critical logical flaw exists in the script's temporal evaluation methodology. The script evaluates the parametric curve strictly at the note's physical start time. For percussive instruments with rapid decay, this is adequate. Yet, for sustained instruments featuring long release envelopes, legato phrasing, or overlapping polyphony, a note's dynamic influence extends far beyond its attack coordinate. While MIDI protocol inherently defines velocity as a singular Note-On event scalar, mapping curves purely by the attack coordinate makes the curve interpolation vulnerable to extreme clustering. A more robust implementation would offer the user the choice to evaluate *t* based on the absolute timeline grid rather than strictly relative note topologies.

2.2 Arbitrary Curve Functions and Lack of Parameterization

The most significant mathematical deficiency in the baseline script lies in its implementation of dynamic curve topologies. The script offers several geometric options, but they are entirely static, depriving the user of the ability to modulate the

rate of change—a feature commonly known as "tension" within professional audio environments.

The "Exponential" curve is hardcoded to evaluate as $t^{2.5}$. While raising a fractional value to an exponent greater than 1.0 mathematically generates a convex ease-in curve, locking the exponent at 2.5 is completely arbitrary. If a composer requires a steeper acceleration profile (e.g., $t^{4.0}$) or a gentler curve (e.g., $t^{1.5}$), the script provides no parameterization to achieve it. The user is inextricably locked into a single logarithmic decay profile.

Similarly, the "Logarithmic" curve utilizes the expression log(1+9t)/log(10), which utilizes a static multiplier of 9 to arbitrarily scale the natural base before converting it via the base-10 logarithm. This creates a fixed concave ease-out curve that cannot be manipulated to fit specific musical phrasing.

Furthermore, the script implements "Sine Wave" and "Cosine Fade" curves utilizing the standard trigonometric functions available in the Python math library. While these are mathematically accurate implementations of basic oscillating functions, pure sinusoidal fades are rarely utilized for isolated volume dynamics in professional mixing contexts because human auditory perception of loudness operates on a logarithmic scale (consistent with the Weber-Fechner law[8]), not a linear or sinusoidal one.

2.3 The Naive Implementation of Stochastic Humanization

To simulate the natural imperfections of human performance, the baseline script includes a "Random" curve type. Because the script must remain self-contained within the FL Studio Python environment—which does not natively guarantee the installation of external data science libraries such as NumPy or SciPy[9]—the author implemented a custom pseudo-random number generator (PRNG) utilizing a classic Linear Congruential Generator (LCG) algorithm:

$$X_{n+1} = (X_n \times 1103515245 + 12345) \bmod 2^{31}$$

This algorithm yields a sequence of pseudo-random integers, which are then divided by 0x7FFFFFFF to output a floating-point number uniformly distributed between 0.0 and 1.0.

While this mathematically satisfies the requirement for random number generation, applying a uniform distribution to MIDI velocity yields acoustically disastrous results in a musical context. A uniform statistical distribution dictates that every value within the defined range has an equal probability of occurrence. However, real human biometric performance does not operate uniformly; it operates on a normal (Gaussian) distribution.[10] A trained concert pianist attempting to strike a chord with a consistent velocity of 80 will execute the majority of keystrokes tightly clustered between 77 and 83. Striking the key with a velocity of 60 would represent a severe mechanical or

[8] Weber-Fechner law: human perception of stimulus intensity follows a logarithmic rather than linear relationship.
[9] FL Studio embeds Python 3.12 in its scripting environment; external pip packages are not guaranteed.
[10] Gaussian distribution: ~68.2% within 1σ, ~95.4% within 2σ, ~99.7% within 3σ of the mean (empirical rule).

neurological error, an event that occurs with exceedingly low probability. By utilizing a uniform LCG algorithm, the baseline script inherently fails to simulate genuine psychomotor jitter, resulting in chaotic, unmusical output rather than organic humanization.

2.4 Rigid Rhythmic Constraints and Metric Ignorance

The script attempts to provide rhythmic utility by including an "Accent Pattern (4/4)" option. The underlying Python logic evaluates the temporal position of the note against the project's internal timing resolution:

$$beat_pos = (note.time \,/\, flp.score.PPQ) \,\%\, 4$$

This specific line of code represents a severe structural vulnerability. The logic relies on modulo 4 arithmetic, operating on the absolute assumption that every musical project is composed in a strict 4/4 time signature and that musical accents must inexorably fall on beats 1, 2, 3, and 4. Furthermore, it assigns predetermined, unchangeable magnitude multipliers to these beats: 1.0 for the downbeat, 0.85 for the third beat, 0.65 for the backbeats, and 0.5 for off-beats.

Music, however, is highly polymorphic and polyrhythmic. If a composer utilizes this script within a composition arranged in a 3/4 waltz, a 5/4 odd-meter groove, or a complex 7/8 progressive sequence, the modulo 4 arithmetic mathematically misaligns the generated accents from the actual musical downbeats.[11] Because the measure lengths do not align with the hardcoded integer of 4, the accents will drift progressively further from the grid with each passing bar, causing destructive rhythmic phasing and applying strong velocities to unintended syncopations.

The flpianoroll API natively exposes global project metadata, specifically score.tsnum (time signature numerator) and score.tsden (time signature denominator).[12] The baseline script's failure to query these properties and adapt its algorithmic behavior dynamically renders the accent feature fundamentally broken for any non-standard musical application.

[11] PPQ (Pulses Per Quarter note) defines the absolute temporal resolution of the FL Studio timeline grid.
[12] flpianoroll.score.tsnum and flpianoroll.score.tsden expose the project time signature numerator and denominator.

3. Advanced Mathematical Paradigms for Continuous MIDI Automation

To elevate the baseline script from a rudimentary utility to a professional-grade DAW composition engine, the arbitrary, static curves must be systematically replaced with sophisticated mathematical models that provide continuous, user-defined parameterization. The most crucial functional requirement is to ensure the script's output perfectly emulates the internal automation curves natively utilized by FL Studio's graphical user interface.

3.1 Transcending Static Exponentials: The FL Studio Tension Algorithm

When users manipulate automation clips, envelope controllers, or Piano Roll event data within FL Studio, they interact with a central "tension" handle. Dragging this handle visually alters the interpolation curve between two control points seamlessly—transitioning from a deeply logarithmic slope, resting at a perfectly linear interpolation at the center, and extending into a deeply exponential slope at the opposite extreme.[13]

To implement this native behavior programmatically within the flpianoroll script, the architecture must introduce a user-defined parameterized tension variable, denoted as v. This variable maps directly to the physical movement of a tension knob, bounded within the continuous interval $[-1.0, 1.0]$. A parameter value of $v = 0.0$ must yield a perfectly linear transition, positive values ($v > 0$) must yield convex (ease-in) geometric shapes, and negative values ($v < 0$) must yield concave (ease-out) geometric shapes.

The fundamental formula requires first mapping the linear user input v into an internal, algorithmically functional tension rate R. The mathematical mapping dictates a piecewise transformation to strictly preserve logarithmic scaling symmetry on both sides of the zero-crossing neutral point:

For convex scaling (Ease-in, v > 0):

$$R = \frac{log_2(v + 1)}{10}$$

For concave scaling (Ease-out, v < 0):

$$R = -\frac{log_2(-v + 1)}{10}$$

For linear scaling (v = 0):

The mathematical curve is purely linear; the function must return the input parameter t unmodified, bypassing the exponential equation to prevent calculation errors.

Once the internal tension rate R is successfully established, the output amplitude y for any given normalized temporal parameter t ∈ [0, 1] is defined by the following core tension function:

$$f(t, R) = \frac{2^{-10Rt} - 1}{2^{-10R} - 1}$$

This specific mathematical architecture ensures that the velocity transitions applied by the Python script will be perceptually and computationally identical to the native visual feedback provided by FL Studio's internal GUI. The denominator serves as a vital normalization constant, ensuring that regardless of the extremity of the tension rate R, the curve will absolutely converge precisely at y = 1.0 when t = 1.0.

4. Formulating the S-Curve: The "Double Curve" Architecture

While a unidirectional tension curve is sufficient for standard fade-ins or fade-outs, professional audio mixing and MIDI orchestration frequently demand sigmoidal geometries. FL Studio's internal toolset explicitly includes a highly utilized "Double Curve" mode, which is fundamentally a piecewise application of the standard tension formula mirrored precisely around the temporal midpoint (t = 0.5).[14]

An S-curve is an absolute necessity for programming natural dynamic swells, such as a cymbal crescendo, a brass section marcato, or a cinematic string rise. In these acoustic scenarios, the physical rate of dynamic change is slow at the onset as energy builds, rapidly and exponentially accelerates through the middle of the phrase, and then smoothly decelerates as it asymptotes toward the target maximum amplitude.

4.1 Evaluation of the Lower Hemisphere

For the first half of the timeline, defined by the condition $0 \leq t \leq 0.5$, the time parameter must be geometrically scaled so that it evaluates the entirety of the tension function within half the physical time span:

$$t' = t \times 2$$

Because the maximum amplitude required at the exact midpoint (t = 0.5) is exactly half of the total output range, the evaluated result of the tension function must be correspondingly divided by two:

$$y = \frac{f(t', R)}{2}$$

4.2 Evaluation of the Upper Hemisphere

For the second half of the timeline, defined by the condition $0.5 < t \leq 1.0$, the curve must smoothly connect with the terminus of the lower hemisphere, but its geometric concavity must be inverted. The tension direction must be reversed to create the mirrored ease-out deceleration effect. First, the local time variable is scaled and inverted:

$$t' = (t - 0.5) \times 2$$

To achieve the geometric inversion, the algorithm evaluates the tension function by subtracting the local time from 1, subtracting that result from 1 to flip the Y-axis projection, dividing by 2 to compress its height, and adding an offset of 0.5:

[14] S-curve / Double Curve: piecewise application of the tension formula mirrored at the temporal midpoint t = 0.5.

$$y = 0.5 + \frac{1 - f(1 - t', R)}{2}$$

S-Curve Hemisphere Evaluation Summary

t Value	Lower Hemisphere	Upper Hemisphere	Perceptual Result
t = 0.25	High compression, low rate of change	N/A	Slow, natural attack profile
t = 0.50	Maximum acceleration	Maximum acceleration	Rapid transition through mid-dynamics
t = 0.75	N/A	High compression, asymptotic deceleration	Smooth landing at target velocity

5. Precision Interpolation via Cubic Bézier Curves

While mimicking the native DAW tension formula is excellent for standard interface emulation, offering full Cubic Bézier curves provides the ultimate industry standard for motion graphics, architectural drafting, and dynamic physics.[15] A Bézier curve allows for independent manipulation of the entry and exit slopes, permitting asymmetrical curves that standard tension formulas cannot replicate.

A standard two-dimensional cubic Bézier curve is defined by four distinct control points in a Cartesian plane: P_0, P_1, P_2, P_3. In the specific context of normalized MIDI velocity mapping, the starting anchor point is fixed at the origin $P_0 = (0, 0)$ and the terminating anchor point is fixed at $P_3 = (1, 1)$. The geometric shape is therefore entirely dictated by the coordinates of the two internal manipulation handles: $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$.

5.1 Bernstein Polynomials and Parametric Equations

The mathematical definition of the curve coordinates (x, y) as a function of an arbitrary interpolation parameter $s \in [0, 1]$ is derived from the foundational Bernstein polynomials.[16] By substituting the fixed anchor coordinates into the generalized Bézier formula, the polynomials reduce to:

$$x(s) = 3(1-s)^2 s \cdot x_1 + 3(1-s)s^2 \cdot x_2 + s^3$$

$$y(s) = 3(1-s)^2 s \cdot y_1 + 3(1-s)s^2 \cdot y_2 + s^3$$

This parametric separation poses a highly significant algorithmic challenge. The flpianoroll script iterates linearly through discrete MIDI notes and calculates a chronological variable t. However, in the Bézier parametric equations, x(s) represents time while y(s) represents the curve's amplitude. The variable s is merely a dimensionless interpolation parameter. Therefore, calculating the correct velocity y at a specific chronological note time t requires inversely solving $x(s) - t = 0$ to find the exact s that produces the known time t.

5.2 Numerical Approximation via the Newton-Raphson Method

Because deriving explicit analytical roots for cubic polynomial equations can introduce complex imaginary numbers and prove unwieldy in standard Python, a highly efficient numerical approximation approach must be utilized. The Newton-Raphson method is exceptionally well-suited for this purpose.[17]

[16] Bernstein polynomials form the mathematical basis for Bézier curve evaluation.
[17] Newton-Raphson method: iterative root-finding algorithm with quadratic convergence for well-behaved functions.

Iterative Refinement:

$$s_{n+1} = s_n - \frac{x(s_n) - t}{x'(s_n)}$$

Derivative Function:

$$x'(s) = 3(1-s)^2 x_1 + 6(1-s)s(x_2 - x_1) + 3s^2(1-x_2)$$

In standard DSP environments, evaluating this Newton-Raphson formula for 3 to 5 iterations guarantees convergence to an extremely accurate floating-point representation. By modifying the ScriptDialog to expose four float input knobs (x_1, y_1, x_2, y_2), the script transitions into a highly customizable, professional-grade shaping tool.

6. Stochastic Biometric Humanization via Gaussian Distributions

A critical secondary application of Piano Roll scripting is algorithmic "humanization"—the intentional and programmatic injection of microscopic timing and velocity imperfections to simulate a live instrumentalist. Digital sequencers are inherently flawless, triggering samples with perfect mathematical rigidity. The baseline script attempts to counteract this by offering a "Random" curve topology, yet its reliance on a uniform generator fundamentally fails to model human psychomotor behavior.

6.1 The Psychoacoustic Flaw of Uniform Randomness

A uniform mathematical distribution assigns equal probability to every value within a specified boundary. If a composer sets a target velocity of 80 with a humanization variance of 20 (range 60–100), the baseline LCG is equally likely to assign 60 as it is 80. Acoustically, this results in rapid, severe, and unnatural dynamic fluctuations. Human musculature and neurological intent operate on a Gaussian distribution. Applying uniform randomness destroys musicality; Gaussian randomness enhances it.

6.2 Implementing the Box-Muller Transform in Native Python

To create genuine biometric jitter, the script must generate normally distributed random variables. The Box-Muller transform[18] converts uniformly distributed variables into standard normal variables ($\mu = 0$, $\sigma = 1$). It requires two independent uniform random variables U_1 and U_2 in (0, 1):

$$Z_0 = \sqrt{(-2\,ln(U_1))} \cdot cos(2\pi U_2)$$

6.3 Algorithmic Application to MIDI Data

The standard normal variable Z_0 must be scaled by the desired variance and shifted to align with the target velocity. Let V_target be the curve velocity at time t, and J be a user-defined Jitter Magnitude parameter (acting as σ):

$$V_{final} = V_{target} + (Z_0 \times J)$$

This framework guarantees precise statistical fidelity: approximately 68.2% of humanized notes fall within ±J of the ideal curve, 95.4% within ±2J, and 99.7% within ±3J.[19] The final output must be clamped to (0.0, 1.0) before assignment to note.velocity.

7. Dynamic Polyrhythmic Scaling via Euclidean Algorithms

The baseline script's hardcoded 4/4 accent pattern represents a severe functional constraint. Music is inherently polymorphic, varying wildly across genres. By natively querying flpianoroll.score properties—specifically score.tsnum and score.tsden—the script can escape the rigidity of modulo 4 mathematics.

7.1 The Mathematical Limitations of Hardcoded Meters

The underlying logic (note.time / flp.score.PPQ) % 4 assumes every measure contains exactly four beats. In a 7/8 time signature, the physical measure lasts 3.5 quarter notes. The hardcoded modulo 4 arithmetic fails to recognize measure boundaries, causing catastrophic rhythmic phasing upon the downbeat of the second measure.

7.2 Integrating Bjorklund's Algorithm for Euclidean Rhythms

Euclidean Rhythms are based on a temporal spacing algorithm originally developed by E. Bjorklund for spallation neutron source accelerators, later adapted for music by Godfried Toussaint.[20] The algorithm distributes k events (accents) as evenly as possible across n total steps.

Bjorklund Algorithm Trace: E(3, 7)

Step	Array State	Operation	Implication
1	[1],[1],[1],[0],[0],[0],[0]	Initialization (k=3, n=7)	Defines accent and rest pools
2	[1,0],[1,0],[1,0],[0]	Append 0s to 1s	Distributes first spacing layer
3	[1,0,0],[1,0],[1,0]	Distribute remainder	Finalizes even spacing
Result	1, 0, 0, 1, 0, 1, 0	Array Concatenation	Valid E(3,7) generated

7.3 Applying Polyrhythmic Scaling to Velocity Geometries

The Euclidean array serves as an algorithmic timing mask. The script calculates the musical step index:

$$step_{index} = \left\lfloor \frac{note.time}{score.PPQ \times \frac{4}{score.tsden}} \right\rfloor mod\ n$$

If euclidean_array(step_index) equals 1, a positive accent scalar (e.g., 1.25) is applied. If 0, a de-accent scalar (e.g., 0.75) is applied. This superimposes mathematically perfect polyrhythms over the continuous tension curve.

8. API Utilization and UI/UX Refinements

The flpianoroll API utilizes the ScriptDialog class to generate custom user interfaces. The current baseline implements only basic AddInputCombo and AddInputKnob controls.

8.1 Expanding the ScriptDialog Framework

To accommodate the proposed mathematical architectures, the createDialog() function must be overhauled:

Tension Parameterization: A continuous float control via form.AddInputKnob('Tension', 0, -1, 1) provides the v parameter for native curve math.

Stochastic Variables: A boolean toggle form.AddInputCheckbox('Enable Jitter', False) and variance control form.AddInputKnob('Jitter Intensity', 0.1, 0.0, 1.0).

Bézier Coordinates: Four float knobs exposing x_1, y_1, x_2, y_2 for custom geometric sculpting.

Target Domain Handling: Query the API via flp.score.getDefaultNoteProperties() for existing states.

8.2 Structural Exception Mitigation

When time_span = 0 (single note or vertically stacked chord), the normalized time variable yields a division by zero. Single-note selections should bypass curve calculations entirely, passing static velocity directly into the Box-Muller jitter or Euclidean accent processors. This ensures mathematical stability regardless of selection footprint.

9. Synthesis of the Refactored Algorithmic Pipeline

Integrating these enhancements requires a strictly ordered programmatic flow within the script's primary apply(form) execution block:

1. State and Context Initialization: Query user parameters from ScriptDialog while fetching flp.score.PPQ, flp.score.tsnum, and flp.score.tsden.

2. Rhythmic Array Pre-computation: If Euclidean scaling is enabled, invoke Bjorklund's algorithm to compute the binary sequence array using the fetched time signature for array length n.

3. Boundary and Domain Mapping: Gather selected notes into a list. Resolve time_min, time_max, and time_span.

4. Note Iteration and Evaluation Loop:

a. Time Normalization — calculate relative parameter t with zero-division safeguards.

b. Curve Application — route t through tension function, Bézier Newton-Raphson solver, or standard curve to produce V_target.

c. Polyrhythmic Scaling — determine Euclidean index via modulo arithmetic; apply accent/de-accent multiplier.

d. Stochastic Jitter — execute Box-Muller transform to inject Gaussian noise at user-defined intensity.

e. Output Clamping — clamp to (0.0, 1.0) and write to note.velocity.

This tightly sequenced pipeline guarantees mathematical precision and prevents sequence degradation, ensuring the user's creative intent is perfectly preserved.

10. Conclusion

The transition from purely visual, manual MIDI editing interfaces to algorithmic, computational generation marks a profound evolution in modern digital music production. The baseline "Velocity Curves" script serves as a functional proof-of-concept for the FL Studio 21.1 Piano Roll API. However, its reliance on elementary arithmetic, static exponential geometry, uniform randomness, and metric rigidity severely limits its professional application.

By systematically substituting static polynomials with the exact non-linear mathematical formulation of FL Studio's native tension parameters, custom scripts achieve seamless continuity with the DAW's established design language. Evolving humanization routines toward Box-Muller Gaussian distributions provides the critical biometric realism necessary to emulate physical instrumentalists. Discarding static beat tracking in favor of dynamic Euclidean rhythm sequences generated by Bjorklund's algorithm allows graceful adaptation to the most complex polyrhythmic compositions.

Implementing these advanced mathematical models refactors the baseline script from a simple utility into a professional-grade dynamic shaping engine, finally realizing the true, uncompromised capability of the embedded Python scripting environment.

Aim Twice, Shoot Once!

Dennis 'dendogg' Smaltz — digiSpace Research